

Server

Storage

- Create Partition

- LVM (Logical Volume Mgmt)

- Disk Usage

File System

- Link

 - Linked Directories

 - Create a link

 - Inode

- Mount

 - /etc/fstab Config Mount

Stratis

SSH, SCP: remote acc

- SSH Agent & Key Forwarding

- Permit Root Login ❌

- Customisation

- SCP: inter-machines file copy

OS ISO Image

- Setup on VMware vSphere

- Bootloader

 - GRUB & GRUB2

 - GRUB

 - GRUB2

- Runlevel: 0-6

HW Check

Kernel & Service

- Kernel

- Service

- Environment Variables

Package Mgmt

Network

- Test Connection

- Network Interface Config

- Bond

- DNS

- nmcli Network Manager

- Firewall

Root / Superuser/ Sudo

- User, Group

 - Special Permissions

ACL

Read files
Write files (ansible, etc.)
 HereDoc
Find files
Process
 Process vs Job:
Utils
Sed
AWK
VIM
Job Scheduler
Monitor
 Datetime
 chrony
Archive
FTP
SELinux
Fine Tune Sys Performance
Network FS
 NFS
 Samba
 NAS: Network Attached Storage
 SAN: Storage Area Network
Podman

Storage

`/dev/sda1` Device SCSI <**device#**(a,b,c)> <**partition#**(1,2,3)>

Mount Points:

- `/` root, for OS per se
- `/customApp` docker, etc
- `swap` usually 2x of mem, or min. 2Gib

Raw: no disk partition, harder partition in the future, e.g. EC2

Create Partition

```
1 fdisk /dev/sdb
2 n # add new partition
3 # partition size = 1e - last sector
4 w # write
```

LVM (Logical Volume Mgmt)

Disk λ-φ mapping, only works after OS installation.

PV: Physical Volume

VG: Volume Group, a set of PV.

LV: Logical Volume

```
1 pvcreate <pv1> <pv2>
2 vgcreate -s 8M <vg> <pv1> <pv2> # -s Phys. Extent (PE) size
3 lvcreate -L <size> -n <lv> <vg>
4 # or by #PEs
5 lvcreate -l <#PE> -n <lv> <vg>
6
7
8 lvdisplay
9
10 mkfs.ext4 <lv_path>
11
12 lvextend -r -L <size (+10G(add), 50G(abs))> <lv_path>
13 # -r: resize FS
```

Disk Usage

```
1 df
2 du /path/to/dir -h | sort -nr | less # -nr: numeric, reverse sort
```

File System

`tree` show folder structure

`cd /` to root dir

`~` home of current user, e.g. `/home/ubuntu`

Extended FS: `ext4`, `xfs`

```
1 mkfs.ext4 /dev/sdc2
```

Link

Linked Directories

```
1 /usr/tmp -> /var/tmp
2 /bin -> /usr/bin
3 /lib -> /usr/lib
```

N.B. `/var/tmp` is diff. from `/tmp`, files in `/tmp` will disappear after reboot.

Create a link

```
ln <target_filepath> <link_name>
```

Link: @ ~/Desktop/ ; Actual file @ /var/tmp/

Write to link will also write to orig. file

```
1 cd /var/tmp/
2 touch file.txt
3
4 cd ~/Desktop/
5 ln -s /var/tmp/file.txt # -s: soft link
6 ls -l
```

- Hard Link: only works w/in same partition
- Soft (Symbolic) Link
- Hard vs. Soft: Soft points to orig. file, Hard points to `inode` in the hard disk, i.e. Hard & Orig. points to the same `inode`.
- *Soft* → Orig. → Inode; *Hard* → Inode

Inode

- var: `nlink` for link count. Inode will be deleted if `nlink` = 0 and file is not opened, i.e. not in RAM.
- Check: `ls -l <filename>` or `stat <filename>`

Mount

```
mount -t <FS_type (ext4, xfs)> <partition> <mount_point>
```

`/etc/fstab` Config Mount

```
1 # <device/UUID> <mount point> <fs_type> <options> <dump> <pass>
2 /dev/sdc2 /home/app ext4 defaults,x-systemd.requires=stratis
```

`<pass>` check FS `fsck`, `1`: root, `2`: others, `0`: no check

```
mount -a
```

`fsck` check FS, i.e. partition, do NOT do this to mounted partition

Stratis

```
1 # Create hard disks on VM manager
2
3 # Create pool
4 stratis pool create <pool> <device1> <device2>
5 stratis pool list
6
7 stratis pool add-data <pool> <device>
8
9 # Create FS
10 stratis filesystem create <pool> <fs>
11 stratis filesystem list
12
13 mkdir <dir>
```

```

14 mount /dev/stratis/<pool>/<fs> <dir>
15 lsblk
16
17 # Create SNAPSHOT
18 stratis filesystem snapshot <pool> <fs> <snapshot_filename>
19
20 # Add entry to /etc/fstab
21 --- /etc/fstab
22 UUID=<UUID> <dir> <fs_type> defaults,x-systemd.requires=stratisd.servic

```

SSH, SCP : remote acc

```
ssh <username>@<[ip | hostname]>
```

Remote acc. w/ SSH key

```

1 # Run on Master
2 ssh-keygen
3 ssh-copy-id <slave_username>@<slave_ip>
4 ssh <slave_username>@<slave_ip>

```

Troubleshoot

```

1 # Login to Slave
2 chmod 700 ~/.ssh
3 chmod 600 ~/.ssh/authorized_keys

```

Master: 192.168.119.217 Slave: 192.168.119.226

Bastion / Jump Host: host created by client, offered to us to run scripts, ansible, etc.

SSH Agent & Key Forwarding

```

1 eval $(ssh-agent) # eval exec arg qua shell cmd.
2 ssh-add ~/.ssh/id_rsa
3
4 ssh -A <username>@<jump-server> # -A: key forward

```

Permit Root Login ❌

```
/etc/ssh/sshd_config.d/00-custom.conf
```

```

1 PermitRootLogin no
2 Port 2222

```

```

1 semanage port -a -t sshd_port_t -p tcp 2222
2
3 firewall-cmd --permanent --add-port=2222/tcp
4 firewall-cmd --reload
5 systemctl restart sshd

```

Customisation

SSHDd locale: `/etc/ssh/sshd_config`

```
1 # Config idle timeout
2 ClientAliveInterval 600 # 10min
3 ClientAliveCountMax 0
4
5 # Disable Root login
6 PermitRootLogin no
7
8 # Disable empty password
9 PermitEmptyPasswords no
10
11 # Limit user SSH acc
12 AllowUsers user1 user2
13
14 # Use different port
15 Port <port#>
16
17 # Locale Setup
18 AcceptEnv LANG LC_CTYPE LC_NUMERIC LC_TIME LC_COLLATE LC_MONETARY LC_ME
19 AcceptEnv LC_PAPER LC_NAME LC_ADDRESS LC_TELEPHONE LC_MEASUREMENT
20 AcceptEnv LC_IDENTIFICATION LC_ALL LANGUAGE
21 AcceptEnv XMODIFIERS
```

Colour for readability in SSH session

Ubuntu

```
1 root@target-machine
2 vi ~/.bashrc
3 force_color_prompt=yes
4
5 source ~/.bashrc
```

SCP : inter-machines file copy

```
1 scp <src> <dest>
2
3 # Local -> Remote
4 scp <filename> <username>@<ip>:<path>
5
6 # Remote -> Local
7 <username>@<ip>:<path> .
```

OS ISO Image

Setup on VMware vSphere

Action → Edit settings

- Hard Disk: Disk Provisioning: *Thin Provision*
- Network: *VM network*
- CD/DVD drive: Datastore ISO file; connected checkbox

- Path: Datastore → SynologyISO → ISO → choose OS

Bootloader

BIOS/UEFI → GRUB(2) → `initrd` → OS kernel (ver) → Modules

GRUB & GRUB2

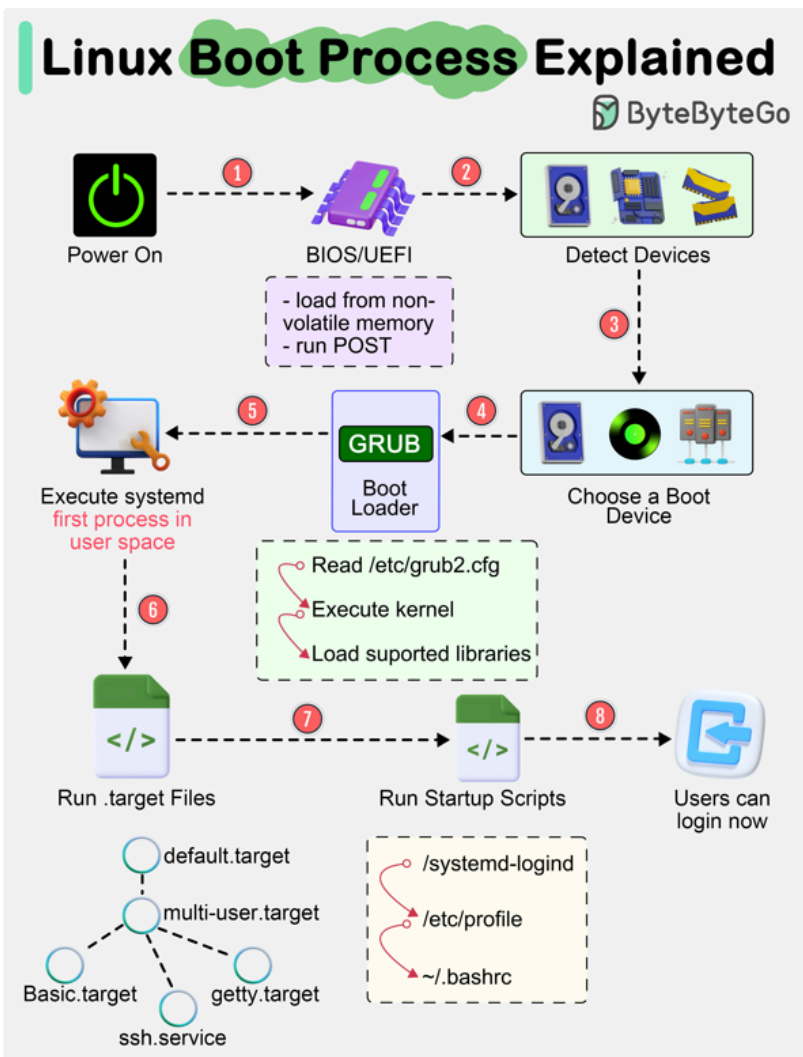
GRUB

- legacy
- `menu.lst`, `grub.conf`
- difficult to modify

GRUB2

- `/etc/default/grub` config here
- `/boot/grub/grub.cfg` read by bootloader, do not modify this file, `/boot/grub2/grub.cfg` for RHEL
- Hidden boot menu (press `Shift`)

Run kernel & `initrd`



Path: `/boot`

`modprobe` : cmd to decide which kernel modules to load / unload, depends on `insmod` .
check `/lib/modules/<module_name>/kernel`

Runlevel: 0-6

- Referred to as targets.
- `systemctl get-default` or `who -r`

HW Check

```
1 | lscpu
2 | lsblk # disks, partitions
3 | lspci
```


Kernel & Service

Kernel

```
1 dmesg | less # Kernel boot log
2 dmesg | grep -i 'pattern' # -i: ignore case sensitivity
3
4 modprobe <module> # load kernel module
```

Service

```
1 systemctl status <service_name>
2 systemctl list-units --type=service
3
4 systemctl enable/disable <service_name> # auto start service on boot
5 systemctl start/stop <service_name>
6
7 systemctl daemon-reload # reload systemd config
8
9 systemctl --user start <service> # allow user to mng service w/o sudo
```

<service-name>.service

```
1 WorkingDirectory=/home/<username>/<proj>
2 # dir abs path
```

Environment Variables

/etc/environment env var.

env get all env var

Write Env var

```
1 export KUBECONFIG="/path/to/yml"
2 echo $KUBECONFIG
3 unset KUBECONFIG
4
5 # Write permanently
6 vi ~/.zshrc
7 --- .zshrc
8 export KUBECONFIG="/path/to/yml"
9
10 source ~/.zshrc
```

All services graceful shutdown before reboot: `shutdown -r now`

Package Mgmt

```
1 apt update && apt upgrade -y
2 # refresh local pkg idx && upgrade installed pkg
```

RHEL, CentOS

```
1 rpm -qa ( | grep <package>) # list package
2
3 dnf update -y
4 dnf remove <package> -y
```

Repos

```
1 cd /etc/yum.repos.d/
2 sudo vi redhat.repo
3 --- redhat.repo
4 [BaseOS]
5 name=BaseOS Repo
6 baseurl=http://content.example.com/rhel9/BaseOS
7 enabled=1
8 gpgcheck=0
9
10 dnf repolist
```

Network

Test Connection

```
1 ping <ip>
2 nc -zv <ip> <port>
3 timeout 2 echo >/dev/tcp/127.0.0.1/9090 && echo "successful" || echo "fa
```

Network Interface Config

Path: `/etc/netplan/`

`00-installer-config.yaml` or `50-cloud-init.yaml`

```
1 ip a # show IP
2
3 # ss: Socket Statistics, replacement of netstat
4 ss -nltp # TCP, Numerical IP, disp Listening skts, show PID.
```

Bond

Path: `/etc/netplan`

Multi NICs (Slave) exist, set mode to increase BW.

Modes

0. balance-rr (Round-Robin)

1. active-backup: 1 active, if it fails, another one will takeover.

2. balancer-xor: connexion based on hash policy, usually replaced by mode 4.
3. broadcast
4. 802.3ad: dynamic link aggregation, if switch supports link aggregation.
5. balance-tlb (Transmit Load Balancing): outbound traffic based on current load, choose the least busy port. No special switch required.
6. balance-alb (Adaptive LB): in & out bound traffic. No special switch required.

Scenario	Mode
----------	------

Simple setup, connect 2 servers directly.	0
---	---

Switch qui support Link Aggregation	4
-------------------------------------	---

Dumb switch	6
-------------	---

`netplan apply` after mod netplan

DNS

`/etc/hosts`

- Mapping IP & DNS
- Pre-DNS lookup
- Higher priority, for DNS blocking. Rules def @ `/etc/nsswitch.conf`

`/etc/resolv.conf`

- If not found in `/etc/hosts`, go for DNS server, def @ `/etc/resolv.conf`
- Priority def at entry: `hosts: files dns` @ `/etc/nsswitch.conf`

`nslookup` name server lookup, query the DNS

`ethtool <intf>`

Network Files & Dir: `/etc/resolv.conf`, `/etc/hosts`, `/etc/hostname`, `/etc/nsswitch.conf`

`nmcli` Network Manager

```

1 hostnamectl set-hostname <server1.example.com>
2
3 nmcli c show # c: connection
4 nmcli c mod <intf ("eth0", "ens192")> ipv4.method manual \
5 ipv4.addr 172.25.250.100/24 \
6 ipv4.gateway 172.25.250.1 \

```

```

7  ipv4.dns 172.25.250.1
8
9  nmcli c mod <intf> +ipv4.addr <IP CIDR> # add IP to intf
10
11 nmcli c up <intf>
12
13 ip a show
14 ip route
15 cat /etc/resolv.conf # DNS
16 hostname

```

IPv4 & Ipv6 adr config stored @ /etc/NetworkManager/system-connections ,
 (/etc/sysconfig/network-scripts/ifcfg-<intf> deprecated)

Firewall

```

1  firewall-cmd --list-all
2
3  firewall-cmd --list-services
4  firewall-cmd --permanent --add-service=<service>
5  firewall-cmd --permanent --add-port=<port#>/<ptcl>
6
7  firewall-cmd --get-active-zones/--get-default-zone
8  firewall-cmd --info-zone=<zone>
9
10 firewall-cmd --get-policies
11 firewall-cmd --info-policy=<policy>
12
13 firewall-cmd --add-rich-rule='rule family="ipv4" source address="<ip>"
14
15 firewall-cmd --reload # IMPORTANT

```

masquerade : in zone & policy info, i.e. NAT

Root / Superuser/ Sudo

Enter root / superuser

```

1  su -
2  su <username> # act as a spec. user
3  sudo su -

```

Modify sudoers list

```

1  su -
2  sudo visudo
3  <username> ALL=(ALL:ALL) ALL
4
5  # Alt
6  usermod -aG wheel <username> # Group: wheel, sudo perm group

```

ALL=(ALL:ALL) ALL : 1e: allow @ any machine, (ALL:ALL) as any user and as any group,
 2e: run all cmd.

User, Group

```
1 useradd <username> -s <login_shell (/bin/bash, /sbin/nologin)>
2 usermod -aG <group1,group2...> <username>
3 userdel -r <username>
4
5 # Add users to group
6 gpasswd -M <user1>,<user2> <group>
7
8 # Remove user from group
9 gpasswd -d <user> <group>
10
11 chage -m <passwd_change_min_intv> -M <passwd_change_max_window> \
12 -E <exp_date> <username>
```

-a : append, -G : secondary groups. only using -G will overwrite user original groups.

! Check available shell: /etc/shells , sbin/nologin ,

Check current shell: echo \$0

Special Permissions

SUID (4): always exec file as Owner User, regardless of the users. Once cmd starts, sys treats proc as if Owner User is running it, pro tem.

-rwsr--r--

```
1 chmod u+s <filename>
```

SGID (2):

- File: being exec as group owner.
- Dir: all files w/in have group ownership of that dir group owner. *For dir qui are used in collab b/w members of a group.*

-rwxrws---

```
1 chmod g+s <filename/dir>
2 # Alt
3 chmod 2770 <filename/dir>
```

Sticky bit (1): only owner & root of a file can delete the file.

```
1 chown -R <username>:<group> file1 file2
```

Show uid , gid , groups

```
1 id <username>
```

/etc/passwd & /etc/shadow: shadow file encrypts the password

/etc/login.defs: def useradd policy

List group members

```
1 getent group <group_name>
```

ACL

```
1 setfacl -m d:u:<username>:<perm> <filepath>
2 setfacl -Rm g:<group>:<perm> <dir>
3 -m: modify
4 -x: remove
5 -R: recursive
6 d: default, new files will inherit ownership attr.
7
8 getfacl <filepath>
```

Read files

```
1 head -n <int> <filename>
2 tail -n <int> <filename>
```

Scroll thru text & Search

```
1 less <filename>
2 # In less:
3 /<keyword> # search keywords in the texts.
```

Write files (ansible, etc.)

```
1 echo "hi" > file.txt # overwrite
2 echo "hi" >> file.txt # append
3 > <filename> # clear file content
4
5 echo "hi" | tee file.txt
6 echo "hi" | tee -a file.txt # append
```

HereDoc

```
1 tee dev_readonly.sql <<-EOF
2 CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL
3 GRANT ro TO "{{name}}";
4 EOF
```

tee read stdin, write it to stdout & file.

- ignore any leading tabs.

EOF : delimiter tk, arbitrary, could be anything. When delimiter tk has quotation marks: "EOF" , it disables variable expansion, i.e. print everything literally, e.g.

```
1 USER="alice"
2 cat <<EOF
3 Hello $USER
4 The time is $(date +%H:%M)
5 EOF
6
7 Output:
8 Hello alice
9 The time is 15:30
10
11 cat <<"EOF"
12 Hello $USER
13 The time is $(date +%H:%M)
14 EOF
15
16 Output:
17 Hello $USER
18 The time is $(date +%H:%M)
```

<dir> usually /home , /usr

xargs : like grep, read items from stdin, and build & exec cmds. e.g. pgrep -f vault |
xargs kill

Find files

```
1 find [path] [options] [expr]
2
3 sudo find / -name *<filename>* 2> /dev/null
4
5 # Find files 10M < size < 100M, GUID set
6 find <dir> -size +10M -size -100M -perm -2000
```

** double-stars qua wildcards to find pattern.

2 File Descriptor for stderr , 2> /dev/null redirect stderr to black hole.

N.B. 2> w/ No Space in-between.

-perm

- mode : match exact mode.
- -mode : at least the spec. bits are set, AND logic, e.g. 2755 matches -2000.
- /mode : any of the target bits, OR logic, e.g. /7000 matchers: 1000, 2000, 4000, etc. /6000 matchers: 2000, 4000, NOT 1000.

Process

```
1 ps aux
2 kill -9 <pid> # -9: force kill
```

- a show proc for all users
- u disc proc user/owner
- x show proc not attached to a terminal.

Process vs Job:

Process: any running program w/ own address space.

Job: concept used by shell. Any program you interactively start that doesn't detach, i.e. not a daemon.

Utils

```
1 echo -ne "$i \t"
2 # -n: do not output the trailing newline
3 # -e: enable interpretation of backslash escapes
4
5 h=$(hostname)
6
7 wc -l # word count #lines
8
9 echo $0 # check current shell
10 echo "" # interpret var: $, cmd sub: `, backslash: \
11 echo ' ' # everything taken literally
12 echo `` # taken as command
```

Sed

Remove lines w/ `##` even w/ whitespaces before it

```
1 sed -i '/^[[[:space:]]*##/d' file.txt
```

AWK

Text processing

NR #records, rows

NF #fields, columns

```
1 awk [opt] 'pattern {action}' input-file > output-file
2 -F: custom field separator, default: space
3 -f: read awk program file
4 -v: assign var before exec
```



```

1 # Unseal Vault
2 keys=$(awk '{print $NF}' keys.txt)
3 for key in $keys; do
4     echo "--- Unsealing w/ key: $key ---"
5     vault operator unseal "$key"
6
7     if vault status | grep -q "Sealed.*false"; then
8         echo "✅ Vault is unsealed!"
9         break
10    fi
11 done
12
13 # Delete multi PVC
14 kubectl -n <ns> get pvc | while read pvc
15 do
16     kubectl -n <ns> delete pvc `echo $pvc | awk '{print $1}' | grep -v N
17 done
18
19 # Parse root_tk.txt to get root token
20 export VAULT_TOKEN=$(awk '{print $4}' root_tk.txt)"

```

```

1 # Replace string
2 awk '{gsub(/gianna/, "Giovanni"); print}' subst.txt > subst1.txt
3
4 # In-place replace edit
5 awk '{gsub(/gianna/, "Giovanni"); print }' file.txt > file.txt.tmp \
6 && mv file.txt file.txt.bak \
7 && mv file.txt.tmp file.txt

```

VIM

```

1 vi file.txt
2 # Cmd mode:
3 :wq -> Enter # Save and Exit
4 gg          # 1e line of doc
5 G          # Last line of doc
6 GA         # Last line and end of line
7 dd         # delete line
8
9 i          # enter insert mode
10 $         # move cursor to the end of line
11
12 /<keyword> # search keyword next / prev by hitting n / Shift+n

```

Job Scheduler

crontab -e open and mod cron schedule file as a user.

```

1 crontab -e
2 * * 29 2 * echo "This cron only runs every min, on 29 Feb"
3 # Cron syntax: <min> <hr> <dayOfMonth> <month> <dayOfWeek[0-7]>

```

at one-off job scheduler

```

1 at <time>
2 commands...
3 Ctrl+D
4

```

```
5 atq      # current queued job
6 atrm <id> # remove job
```

Monitor

uptime uptime & load average in 1, 5, 15 min, often cf. #cores.

Waiting tasks = load_avg - (#cores * util_CPU), **uptime** - (**nproc** * **top**).

top real-time proc. monitor

free check free mem & swap space.

tcpdump network packet analyser, ~wireshark.

Limit sys log size: **systemd-journald.service** & **/etc/systemd/journald.conf**

```
1 --- /etc/systemd/journald.conf
2 SystemMaxUse=100M
3
4 systemctl restart systemd-journald
```

Datetime

date

timedatectl

chrony

/etc/chrony.conf

```
1 server pool.ntp.org iburst
```

```
1 systemctl restart chronyd
2 systemctl enable chronyd
```

chronyc chrony CLI

Archive

```
1 tar --gzip -cf /tmp/backup.tar.gz /etc/sysconfig
2 # -c: create, -x: extract, -f: spec filename (must be last option), -j:
```

FTP

Server

```
1 vi /etc/vsftpd/vsftpd.conf
2 --- vsftpd.conf
3 anonymous_enable=NO
4 ascii_upload_enable=YES
5 ascii_download_enable=YES
6 use_localtime=YES
7
8
9 systemctl start vsftpd
10 systemctl stop firewalld
```

Client

```
1 ftp <ftp_server_ip>
2 # Enter server username & password
3 bin
4 hash
5 put <filename>
6 # Check server user's home dir
```

SELinux

`sestatus` check SELinux status

Label: `user:role:type:level`

```
1 ls -lZ
2 ls -dZ
```

Config

```
1 /etc/selinux/config
2 touch /.autorelabel
```

```
1 # Boolean: run-time switch to enable/disable SELinux policy
2 getsebool -a
3 setsebool -P <boolean_name> on
4
5 # Change type label
6 chcon -t <type> <filename (e.g. "/var/www/custom(/.*)?")>
7 semanage fcontext -a -t <type> <filename>
```

Troubleshoot for SELinux denial

```
1 ausearch -m avc # avc: Access Vector Cache
```

Fine Tune Sys Performance

`tuned`

```
1 tuned-adm active
2 tuned-adm list
3 tuned-adm profile <profile> # change to desired profile
```

```
4 tuned-adm recommend
5 tuned-adm off
```

nice & renice Prioritise process

Check sys priority: **top**

PR process actual priority: RT-39 (RT, -99, -98,...; HI-LO)

NI user space process priority: -20-19 (HI-LO), mapping 0-39 of **PR**, $PR = NI + 20$

```
1 ps axo pid,comm,nice,cls --sort-nice
2
3 # start command w/ proc prior
4 nice -n <prior#> <pname>
5
6 # change proc prior (running)
7 renice -n <prior#> <pid>
```

Network FS

NFS

- Linux to Linux
- Matching UID/GID

Server side

```
1 dnf install nfs-utils -y
2
3 systemctl enable --now nfs-server
4
5 mkdir <server_dir>
6 chmod -R 777 <server_dir>
7
8 --- /etc/exports
9 <server_dir> <client_ip>(rw,sync,no_root_squash)
10 # Root squash: map client root user (UID 0) to a less privileged user
11
12 exportfs -rv
13
14 firewall-cmd --add-service=nfs --permanent
15 firewall-cmd --reload
```

Client side

```
1 dnf install nfs-utils -y
2
3 mkdir <client_dir>
4
5 --- /etc/fstab
6 <server_ip>:<server_dir> <client_dir> nfs _netdev,vers=4.2,rw 0 0
7
8 sudo systemctl daemon-reload
9 sudo mount -a
```

Alt: **autofs**

```

1 dnf install autofs -y
2
3 vi /etc/auto.master
4     /mnt/test /etc/auto.nfs
5
6 vi /etc/auto.nfs
7     <subdir> -rw,sync <server_ip>:/nfs/share # /mnt/test/<subdir>
8
9 systemctl restart autofs
10 systemctl enable autofs
11 systemctl status autofs
12
13 cd /mnt/test/<subdir>

```

Samba

- Windows or mix environments
- Rely on user, password, AD
- Label path on SELinux

Server side

```

1 dnf install samba polycoreutils-python-utils -y
2
3 --- /etc/samba/smb.conf
4 [smbashare]
5     path = <server_dir>
6     browsable = yes
7     guest ok = yes
8     read only = no
9
10
11 testparm -s # check & verify smb.conf
12 semanage fcontext -a -t samba_share_t "<server_dir>(/.*)"
13 restorecon -Rv <server_dir>
14
15 firewall-cmd --add-service=samba --permanent
16 firewall-cmd --reload
17
18 systemctl enable --now smb

```

Client side

```

1 dnf install samba-client cifs-utils -y
2
3 smbclient -L //<server_ip>/<server_dir> -N
4
5 get <filename>
6
7 mount -t cifs //<server_ip>/<server_dir> <client_dir> -o guest
8
9 --- /etc/fstab
10 //<server_ip>/<server_dir> <client_dir> cifs _netdev,guest 0 0
11
12
13 mount -a

```

NAS: Network Attached Storage

- a device as a node spec for storage.

- File-level acc, e.g. data requires protection.
 - cf. Block-level and Object-level: unstructured data, large dataset
- Employ NFS or SMB
- TCP/IP ethernet Packets: latency

SAN: Storage Area Network

- Conn. storage devices, separated from LAN
- Block-level acc: seq. of bytes having whole #records, e.g. DB, RAID
- Fibre channel

Podman

```
1 | podman info # check env, registry/repo info
```